

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Computer Network Lab (23CS5PCCON)

Submitted by

AFEEFAH (1BM23CS015)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

September 2025 – January 2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Computer Network (23CS5PCCON)” carried out by **AFEEFAH (1BM23CS015)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above-mentioned subject and the work prescribed for the said degree.

Praveen N Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	---

Index

Sl. No.	Date	Experiment Title	Page No.
1	21-08-2025	Create a topology and simulate sending a simple PDU from source to destination using hub and switch as connecting devices and demonstrate ping message	2-4
2	28-08-2025	Configure DHCP within a LAN and outside LAN.	5-6
3	04-09-2025	Configure Web Server, DNS within a LAN.	8-10
4	04-09-2025	Configure IP address to routers in packet tracer. Explore the following messages: ping responses, destination unreachable, request timed out, reply	11-12
5	11-09-2025	Configure default route, static route to the Router	13-17
6	09-10-2025	To understand the operation of TELNET by accessing the router in server room from a PC in IT office.	18-20
7	16-10-2025	Configure RIP routing Protocol in Routers.	21-23
8	16-10-2025	To construct a WLAN and make the nodes communicate wirelessly.	24-25
9	16-10-2025	To construct simple LAN and understand the concept and operation of Address Resolution Protocol (ARP).	26-27
10	30-10-2025	To construct a VLAN and make the PC's communicate among a VLAN.	28-30
11	06-11-2025	Configure OSPF routing protocol.	31-35
12	13-11-2025	Write a program for congestion control using Leaky bucket algorithm.	36-37
13	13-11-2025	Write a program for error detecting code using CRC-CCITT (16-bits).	38-42
14	20-11-2025	Using TCP/IP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.	43-45
15	20-11-2025	Using UDP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.	46-50

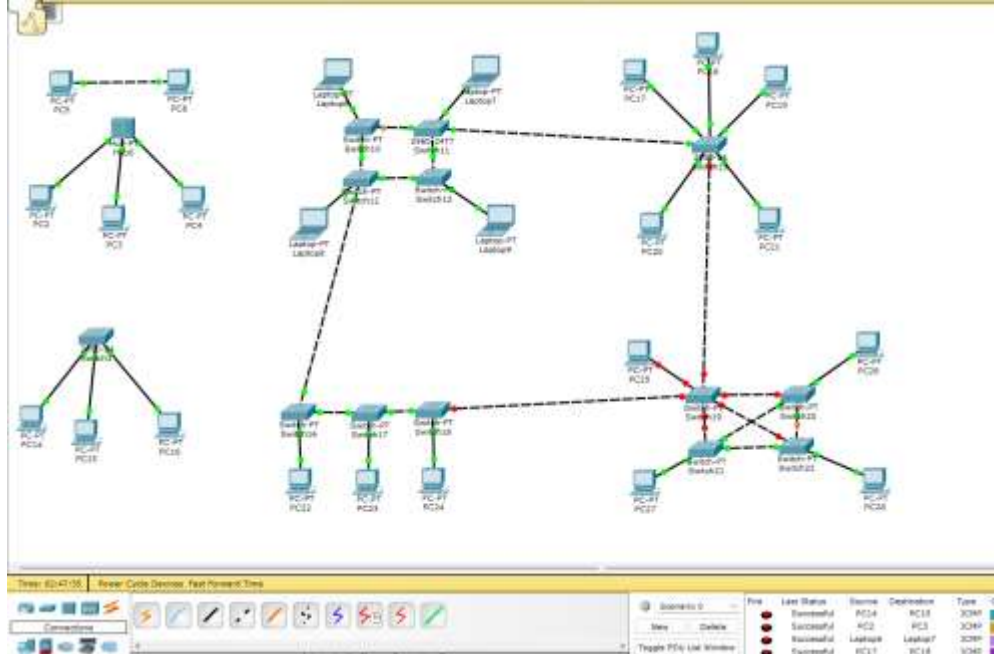
Github Link:

<https://github.com/Afeefahafeefah/5th-SEM-LABS/CN LAB>

Program – 1:

Aim: Create a topology and simulate sending a simple PDU from source to destination using hub and switch as connecting devices and demonstrate ping message.

Topology:



Procedure:

- **Point-to-Point link**
 - PC5 → 20.10.0.2
 - PC6 → 20.10.0.3
- **Hub (PC2, PC3, PC4)**
 - PC2 → 20.10.0.4
 - PC3 → 20.10.0.5
 - PC4 → 20.10.0.6
- **Switch (PC14, PC15, PC16)**
 - PC14 → 20.10.0.7
 - PC15 → 20.10.0.8
 - PC16 → 20.10.0.9

LAN Topology & PC IP Configurations

1. Star Topology (Switch13 Region)

- PC17 → 10.0.0.2
- PC18 → 10.0.0.3
- PC19 → 10.0.0.4
- PC20 → 10.0.0.5
- PC21 → 10.0.0.6

2. Bus Topology (Switch16 → Switch17 → Switch18)

- PC22 → 10.0.0.7
- PC23 → 10.0.0.8
- PC24 → 10.0.0.9

3. Ring Topology (Switch19 → Switch20 → Switch21 → Switch22)

- PC25 → 10.0.0.10
- PC26 → 10.0.0.11

- **PC27** → 10.0.0.12
- **PC28** → 10.0.0.13

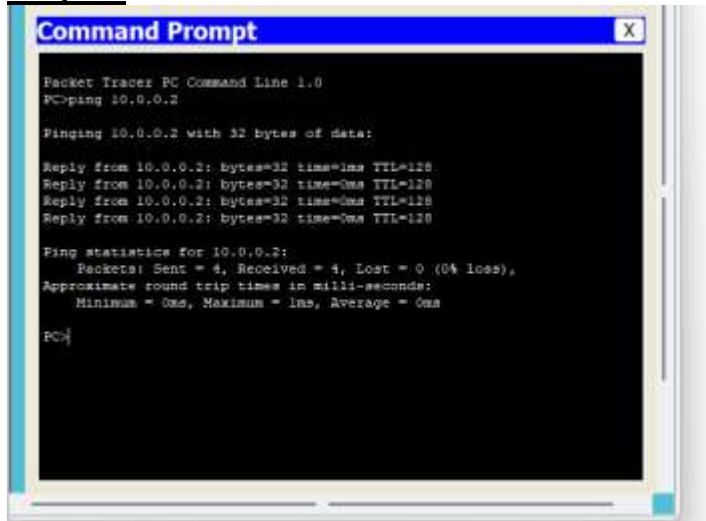
4. Mesh Topology (Bottom-right interconnected switches)

If mesh includes the same PCs (25–28), then **IPs remain the same**.

If you have more PCs, assign next IPs:

- **Next PC** → 10.0.0.14
- **Next PC** → 10.0.0.15
- Continue as needed...

Output:



```

Packet Tracer PC Command Line 1.0
PC>ping 10.0.0.2

Pinging 10.0.0.2 with 32 bytes of data:

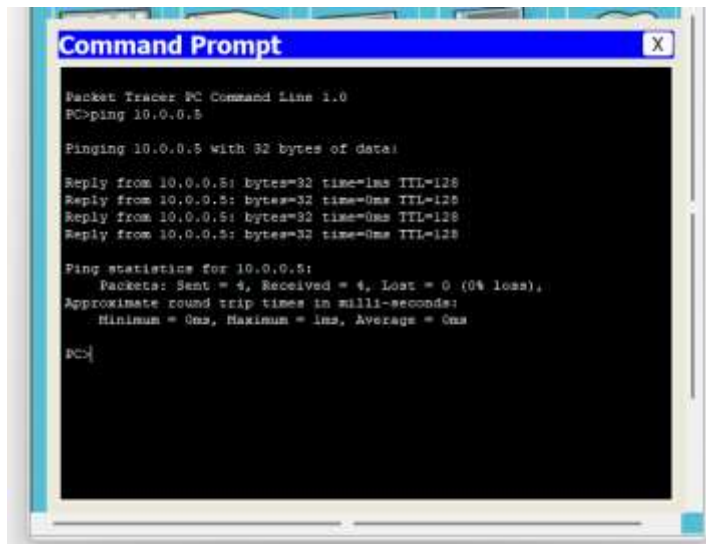
Reply from 10.0.0.2: bytes=32 time=1ms TTL=128
Reply from 10.0.0.2: bytes=32 time=0ms TTL=128
Reply from 10.0.0.2: bytes=32 time=0ms TTL=128
Reply from 10.0.0.2: bytes=32 time=0ms TTL=128

Ping statistics for 10.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

PC>

```

Fig 1.1 Command Prompt of PC22



```

Packet Tracer PC Command Line 1.0
PC>ping 10.0.0.5

Pinging 10.0.0.5 with 32 bytes of data:

Reply from 10.0.0.5: bytes=32 time=1ms TTL=128
Reply from 10.0.0.5: bytes=32 time=0ms TTL=128
Reply from 10.0.0.5: bytes=32 time=0ms TTL=128
Reply from 10.0.0.5: bytes=32 time=0ms TTL=128

Ping statistics for 10.0.0.5:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

PC>

```

Fig 1.2 Command Prompt of PC26

Observation:

Lab-1

Interconnection of diff. topology → Hybrid
Topology :- architectural of cables, computer
Topology types star, bus, ring, mesh

Bus :- All computers & devices are connected to a single main cable
Star :- All devices are connected to a central device as hub.

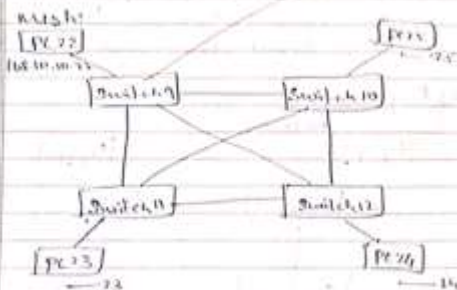
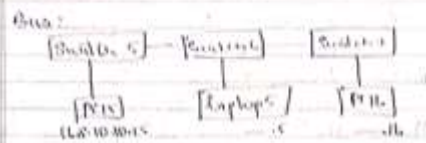
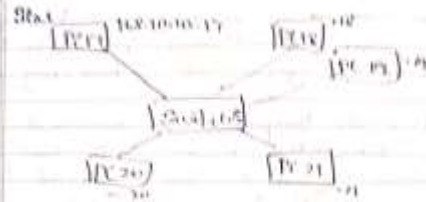
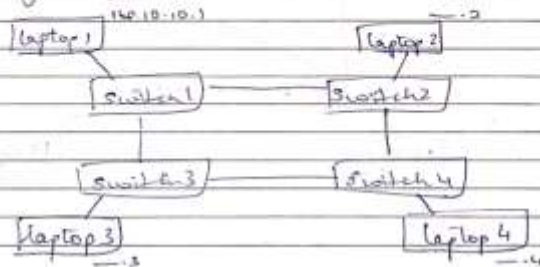
Ring :- Each device is connected exactly to two other devices forming a circular path.

Mesh :- Every device is connected to every other device directly.

Hub :- Its network device that connects multiple computer in a star topology.

Switch :- Connects to multiple computers in a network & sends data only to the device its meant for

Ring :-



Hybrid

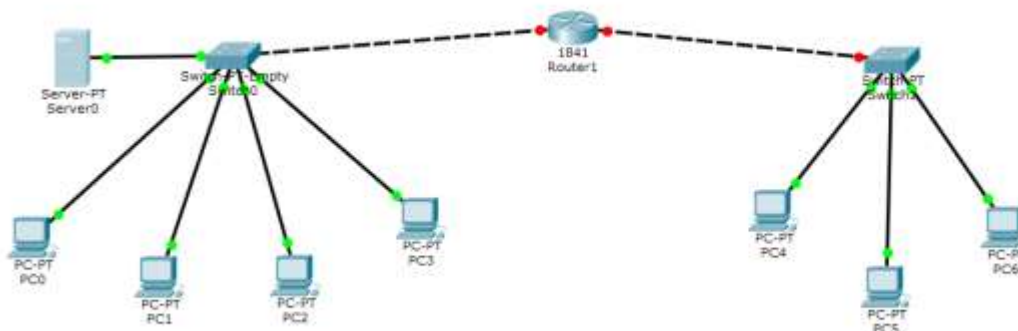


Hybrid

Program – 2:

Aim: Configure DHCP within a LAN and outside LAN.

Topology:



Procedure:

1. Configure DHCP Server

1. On the Server → **Desktop > IP Configuration**, set:
 - **Static IP:** 192.168.10.2
 - **Default Gateway:** 192.168.10.1
2. Open **Services > DHCP** and add the following two DHCP pools:

(a) Pool Name: switch1

- Gateway: 192.168.10.1
- Start IP: 192.168.10.3
- Subnet Mask: 255.255.255.0
- Max Users: 20

(b) Pool Name: switch2

- Gateway: 192.168.20.1
- Start IP: 192.168.20.2
- Subnet Mask: 255.255.255.0
- Max Users: 20

3. Turn **DHCP Service ON**.

4. Go to **Config > Interface > FastEthernet0** and assign server IP:
 - IP Address: 192.168.10.10
 - Port Status: ✓ ON

2. Configure Router

i. Router > **enable**

ii. Router# **configure terminal**

(Within LAN)

iii. Router(config)# **int fa0/0**

iv. Router(config-if)# **ip address 192.168.10.1 255.255.255.0**

v. Router(config-if)# **ip helper-address 192.168.10.2**

vi. Router(config-if)# **no shutdown**

vii. Router(config-if)# **exit**

(Outside LAN)

viii. Router(config)# **int fa0/1**

ix. Router(config-if)# **ip address 192.168.20.1 255.255.255.0**

x. Router(config-if)# **ip helper-address 192.168.10.2**

xi. Router(config-if)# **no shutdown**

xii. Router(config-if)# **exit**

xiii. Router(config)# **exit**

xiv. Router# **write memory**

Output:



Fig 2.1 DHCP Inside LAN

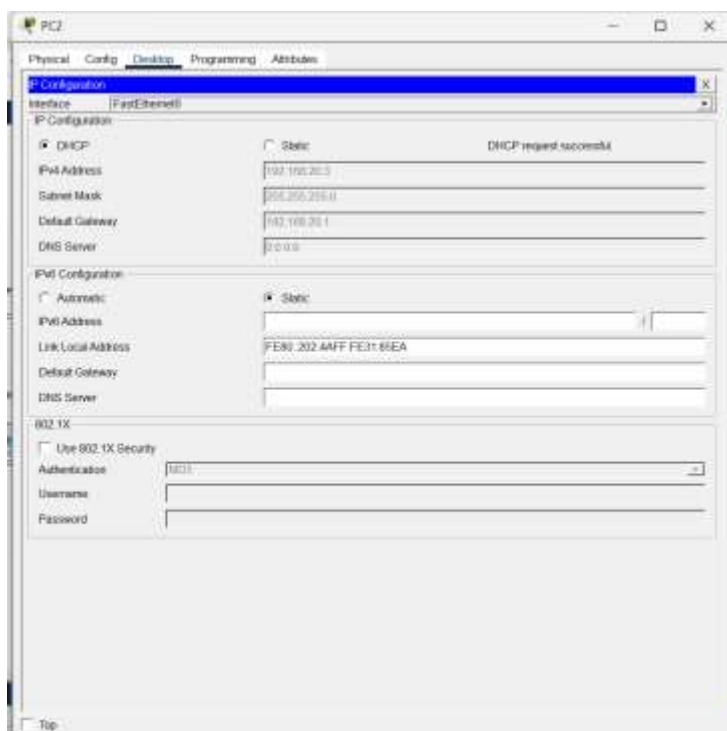


Fig 2.2 DHCP Outside LAN

Observation:

LAB-2

Configure DHCP within a LAN & DHCP

ADP - server fails to connect to a LAN
because it gives a random (wrong) IP
so if fails
This happens when server configuration is
changed into DHCP

→ click on server server → setting → DHCP
→ give poolname as lab2
→ Start IP address: 192.168.10.1
→ No. of users: 12
→ click on save

go config → Interface → Fast Ethernet
from static to DHCP

click on PO → IP config → DHCP
do for all PCs

↳ DHCP server request success

Outside LAN
normally connect
then add a

DHCP

Initiation a new pool name ()
IP address: 192.168.10.1
Subnet mask: 255.255.255.0
No. of users: 13
Add it

Desktop → IP config → static → IP address
192.168.10.1

subnet mask: 255.255.255.0
Default gateway: 192.168.10.1

gateway: 192.168.10.1
IP address: 192.168.10.3
subnet mask: 255.255.255.0
users: 20

11/2

pool name: switch two
gateway: 192.168.10.1

IP address: 192.168.10.2
subnet mask: 255.255.255.0
users: 20

click on router → CLI → no → enter → enter
then Router > enable → conf t → fasteth
Router (config) # int fa 0/0
↳ means interface

Router (config-if) # ip address 192.168.10.1
255.255.255.0

Router (config-if) # ip helper address
192.168.10.2

Router (config-if) # no shutdown
(on entire this)

→ do write memory
→ gives as → building configuration...

→ Router (config-if) # exit

for new switch:

→ Router (config) # int fa 0/1

→ Router (config-if) # ip address 192.168.10.2
255.255.255.0

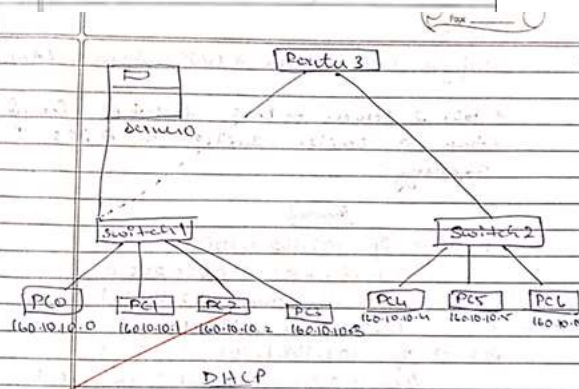
→ Router (config-if) # ip helper address
192.168.10.2

→ Router (config-if) # no shutdown

→ Router (config-if) # exit

→ Router (config) # do write memory

building configuration...
[OK]

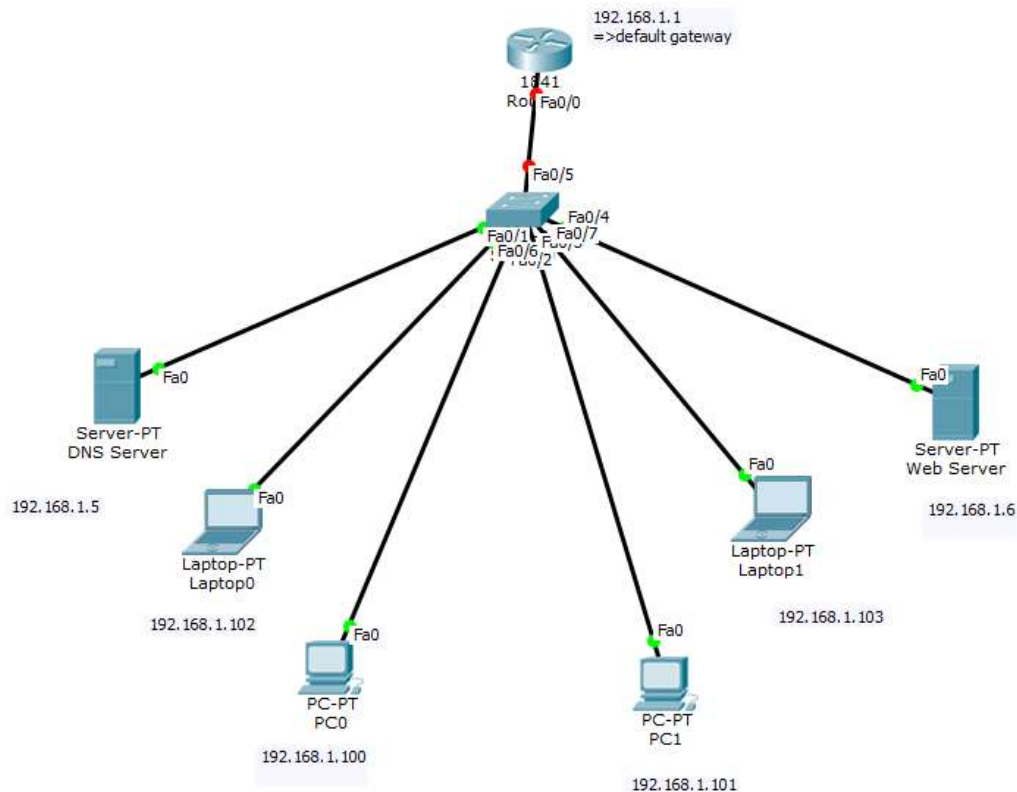


21/10/2020

Program – 3:

Aim: Configure Web Server, DNS within a LAN.

Topology:



Procedure:

1. PC Configurations

PC0

- IP Address: **192.168.1.100**
- Subnet Mask: **255.255.255.0**
- Default Gateway: **192.168.1.1**
- DNS: **192.168.1.5**

PC1

- IP Address: **192.168.1.101**
- Subnet Mask: **255.255.255.0**
- Default Gateway: **192.168.1.1**
- DNS: **192.168.1.5**

2. Servers Required

- **DNS Server**
- **Web Server**

3. Configure Web Server

1. Open **Server > Services > HTTP**
2. Turn **HTTP** and **HTTPS** ON
3. Go to **Desktop > Edit HTML**
 - Edit the default HTML page to display a custom message
4. Assign IP to Web Server:

- **IP:** 192.168.1.6
- **Subnet Mask:** 255.255.255.0
- **Default Gateway:** 192.168.1.1
- **DNS:** 192.168.1.5

4. Configure DNS Server

1. Go to **Services > DNS**
2. Add a new DNS entry:
 - **Name:** www
 - **Domain:** letslearn.com
 - **Address:** 192.168.1.6 (IP of web server)
3. Ensure DNS Service is **ON**
4. DNS Server IP configuration:
 - **IP:** 192.168.1.5
 - **Subnet Mask:** 255.255.255.0
 - **Default Gateway:** 192.168.1.1
 - **DNS:** 192.168.1.5

5. Check Connectivity

On PC → **Command Prompt**

- ping 192.168.1.5 → tests DNS server connection
- ping 192.168.1.6 → tests web server connection

Open **Desktop > Web Browser** and enter:

http://www.letslearn.com

The webpage should load successfully.

6. Extra Tasks

- Modify the HTML page and verify changes appear in browser
- Add more PCs and configure their IP, gateway, and DNS similarly

Output:



Fig 3.1 Browser search output

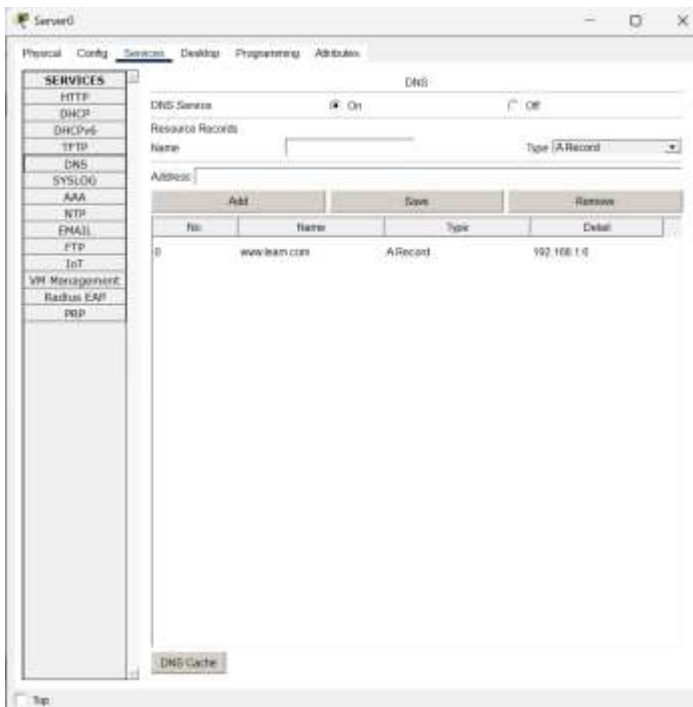


Fig 3.2 DNS Server Records

Observation:

Lab 3

Configure two servers & DNS within a LAN

Take 2 servers → DNS & web server
also a router, switch & 2 PCs

Topology:

Router
IP: 192.168.1.100
Subnet mask: 255.255.255.0
Default gateway: 192.168.1.1
DNS server: 192.168.1.5

PC2 → IP: 192.168.1.101
Default gateway: Same as connected to the same router

Web server → Services → HTTP
to make sure HTTP & HTTPS are 'on'.
→ select any file in file manager
→ click on edit
→ edit the file → Save → yes

Web server → desktop → IP config → static
IP: 192.168.1.6
Default gateway: 192.168.1.1
DNS server: 192.168.1.5

DNS server → Services → DNS is 'on'
Name: www.lulu.com
Type: A Record
Address: 192.168.1.5 → (IP of web server)

PC0 → Desktop → Command prompt
→ Ping 192.168.1.5
Web server → Desktop → web browser
to access lulu.com

DNS server
IP: 192.168.1.5
255.255.255.0
192.168.1.1
192.168.1.5

Router 192.168.1.1

Topology:

```

graph TD
    Router[Router 192.168.1.1] --- Switch[Switch]
    Switch --- PC0[PC0]
    Switch --- PC1[PC1]
    Switch --- WebServer[Web Server]
    Switch --- DNSServer[DNS Server]
  
```

Router 192.168.1.1

Switch

PC0

PC1

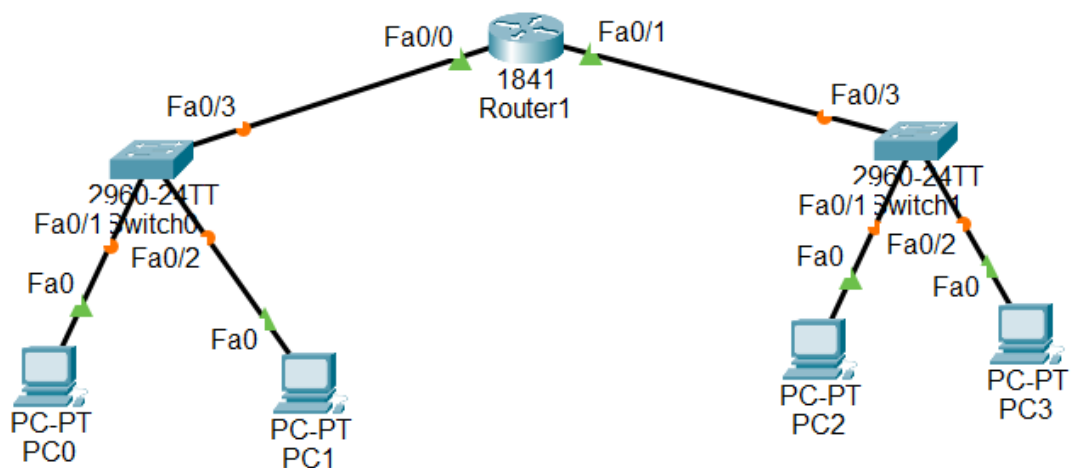
Web Server

DNS Server

Program – 4:

Aim: Configure IP address to routers in packet tracer. Explore the following messages: ping responses, destination unreachable, request timed out, reply

Topology:



Procedure:

1. Network Setup

Two PCs connected through a switch and router.

- **PC0 IP:** 192.168.2.100
- **PC1 IP:** 192.168.2.101

Output:

```
PC>ping 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

PC>ping 192.168.1.100

Pinging 192.168.1.100 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.
```


Observation:

pc> ping 192.168.2.101

Pinging 192.168.2.101 with 32 bytes of data:

Reply from 192.168.2.101: ...

Reply from 192.168.2.101: ...

Reply from 192.168.2.101: ...

Reply from 192.168.2.101: ...

This indicates successful connectivity.

pc> ping 192.168.2.10

Pinging 192.168.2.10 with 32 bytes of data:

Request timed out.

Request timed out.

Request timed out.

Request timed out.

Meaning: No device with this IP is present in the network.

8. Configure IP addresses to router in packet traces, explore the following responses:

- i> ping messages
- ii> destination unreachable
- iii> requests timed out
- iv> reply

1. Connect router to switch

```
graph TD
    Router[Router 192.168.1.1] --- Switch[Switch]
    Switch --- PC1[PC1 192.168.10.100]
    Switch --- PC2[PC2 192.168.1.101]
```

Ping (Router IP) → 192.168.1.1

3. Disconnect router from topology & ping router from PC

Ping 192.168.1.1

request timed out.

4. Reply from 192.168.1.1 Bytes = 32

Time	TTL
Time = 1ms	TTL = 225
Time = 0ms	TTL = 225
Time = 2ms	TTL = 225

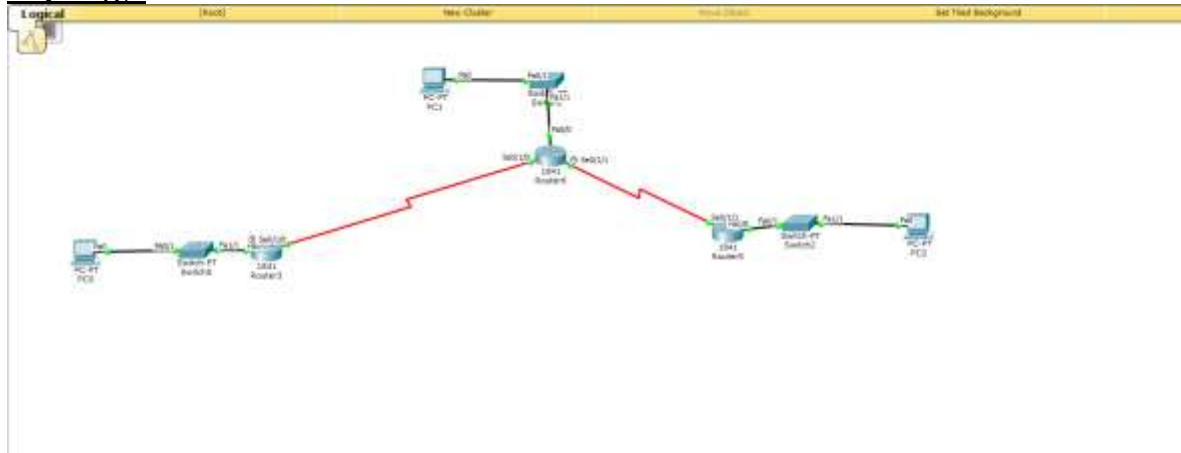
2. Destination host unreachable after disconnecting router with switch

Ping 192.168.1.1

Program – 5:

Aim: Configure default route, static route to the Router

Topology:



Procedure:

1. Create Topology

Create a 3-router topology where each topology contains:

- 1 PC
- 1 Switch
- 1 Router

2. Router Hardware Setup

For **all 3 routers**:

- Click on **Physical** tab
- Turn **Router OFF**
- Drag & drop **HWIC-2T module**
- Turn **Router ON**

3. Connect the Routers

Use **Serial DCE cable** to interconnect all the 3 routers in serial topology.

Router Configurations:

Router 1

Step 1: Configure Serial Interface

```
Router> enable
Router# configure terminal
Router(config)# int Se0/1/0
Router(config-if)# ip address 172.16.1.1 255.255.255.252
Router(config-if)# no shutdown
Router(config-if)# exit
```

Step 2: Configure FastEthernet

```
Router(config)# interface Fa0/0
Router(config-if)# ip address 192.168.10.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
```

Step 3: Save Configuration

```
Router# write memory
Router# exit
```

Router 2

```
Router> enable
```

```
Router# configure terminal
Router(config)# hostname R2
Serial Interface (to Router 1)
R2(config)# int Se0/1/0
R2(config-if)# ip address 172.16.1.2 255.255.255.252
R2(config-if)# no shutdown
FastEthernet Interface
R2(config)# int Fa0/0
R2(config-if)# ip address 192.168.20.1 255.255.255.0
R2(config-if)# no shutdown
Serial Interface (to Router 3)
R2(config)# int Se0/1/1
R2(config-if)# ip address 172.16.2.1 255.255.255.252
R2(config-if)# no shutdown
R2(config-if)# exit
R2# write memory
```

Router 3

```
Router> enable
Router# configure terminal
Router(config)# hostname R3
Serial Interface (to Router 2)
R3(config)# int Se0/1/1
R3(config-if)# ip address 172.16.2.2 255.255.255.252
R3(config-if)# no shutdown
FastEthernet Interface
R3(config)# int Fa0/0
R3(config-if)# ip address 192.168.30.1 255.255.255.0
R3(config-if)# no shutdown
R3(config-if)# exit
R3# write memory
```

PC IP Configuration

PC0

- IP: 192.168.10.10
- Default Gateway: 192.168.10.1

PC1

- IP: 192.168.20.10
- Default Gateway: 192.168.20.1

PC2

- IP: 192.168.30.10
- Default Gateway: 192.168.30.1

Static Route Configuration

Router 1

```
R1> enable
R1# configure terminal
R1(config)# hostname R1
R1(config)# ip route 192.168.20.0 255.255.255.0 172.16.1.2
R1(config)# ip route 192.168.30.0 255.255.255.0 172.16.1.2
R1# write memory
```

Router 2

```
R2> enable
R2# configure terminal
```



```

R2(config)# ip route 192.168.10.0 255.255.255.0 172.16.1.1
R2(config)# ip route 192.168.30.0 255.255.255.0 172.16.2.2
R2# write memory

```

Router 3 (Default Route)

```

R3> enable
R3# configure terminal
R3(config)# ip route 0.0.0.0 0.0.0.0 Se0/1/1
R3# write memory

```

Output:

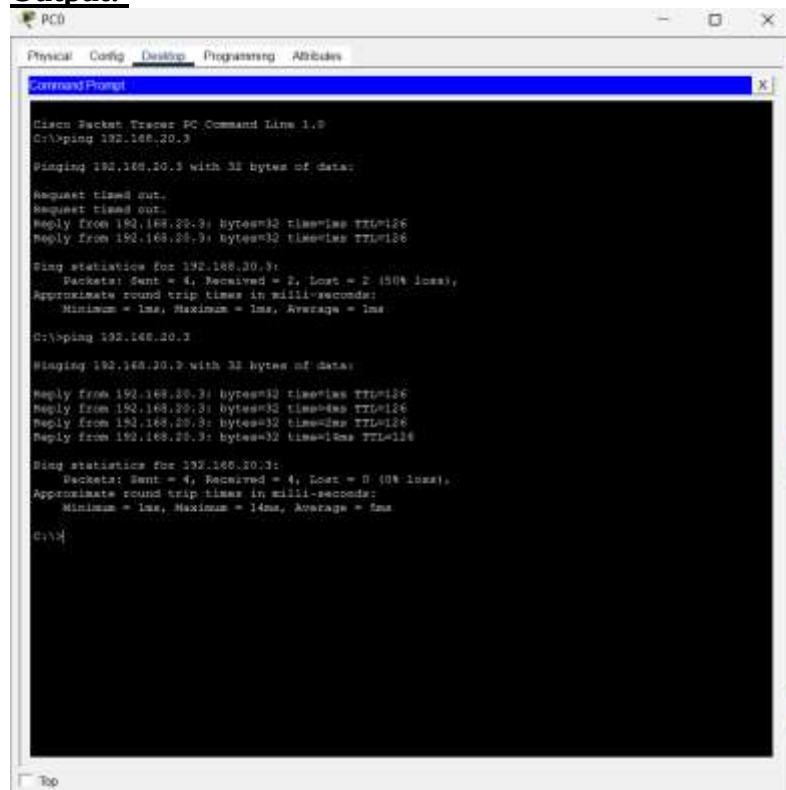


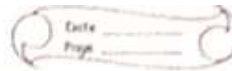
Fig 5.1 ping to PC1 from PC0



Fig 5.2 ip route information in default router

Observation:

Lab - 4



1. Configure default route, static route to the router.

- Take

* we use Serial Interface cable to connect routers.

Click on router → is configured for "if not Physical" → turn off router → select HWIC-2T → drag & drop on router.
→ turn on the router again!

* Connect the ~~cable~~ routers through 'Serial DCE' se 0/1/0 , se 0/1/1

* click on router → CLI → "no"

→ "enable" ; then → "conf t"

→ int se 0/1/0

#ip address 172.16.1.1 255.255.255.252

#no shutdown

#exit

#interface fa0/0

#ip address 192.168.10.1 255.255.255.0

#no shutdown

#exit

→ address cli + 2.

wr

Building Configuration...

[OK]

Configuring R2.
 → no, press ENTER
 > enable
 # conf t
 # hostname R2
 # int se 0/0/0
 # ip address 172.16.1.2 255.255.255.252
 # no shutdown
 exit
 # int se 0/1/1
 # ip address 172.16.2.1 255.255.255.252
 # no shutdown
 exit
 # press ctrl+z
 → wr
 Building configuration...
 [ok]

Configuring the PCs
 PC0: IP: 192.168.10.10
 Default Gateway: 192.168.10.1
 PC1: IP: 192.168.20.10
 DG: 192.168.20.1
 PC2: IP: 192.168.30.10
 DG: 192.168.30.1
 DNS: 0.0.0.0

Go to Router 1:
 > enable
 # conf t
 # hostname R1
 # ip route 192.168.20.0 255.255.255.0 172.16.2.1
 # ip route 172.16.2.0 255.255.255.0 172.16.1.2
 # ip route 192.168.30.0 255.255.255.0 172.16.1.1
 → press ctrl+z
 wr

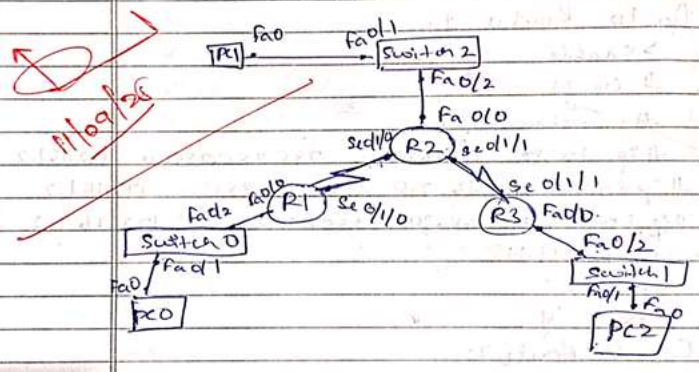
Go to Router 2:
 # enable
 # conf t
 # ip route 192.168.10.0 255.255.255.0 172.16.1.2
 # ip route 192.168.30.0 255.255.255.0 172.16.2.1
 → press ctrl+z, wr

Go to Router 3:
 # enable, # conf t
 # ip route 0.0.0.0 0.0.0.0 se 0/1/0
 → press ctrl+z, wr.

Go to PC0 & ping:
 > ping 192.168.10.1
 > ping 192.168.20.1
 > ping 192.168.30.1

then send message:
 PC0 → PC1
 PC2 → PC2
 PC2 → PC1

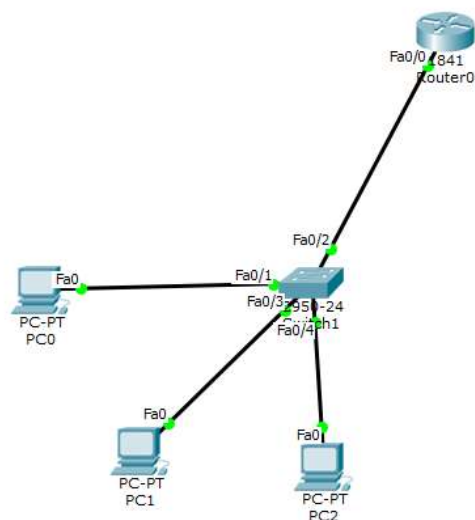
Successful



Program – 6:

Aim: To understand the operation of TELNET by accessing the router in server room from a PC in IT office.

Topology:



Procedure:

1. Topology

- PC0 → Switch0 → Router0

2. PC0 IP Configuration

- **IP Address:** 192.168.1.2
- **Default Gateway:** 192.168.1.1

3. Configure Router for Telnet

Open CLI on Router0:

Router> enable

Router# configure terminal

Router(config)# hostname R1

Assign IP Address to Router Interface

R1(config)# interface Fa0/0

R1(config-if)# ip address 192.168.1.1 255.255.255.0

R1(config-if)# no shutdown

R1(config-if)# exit

4. Configure Telnet (VTY lines)

R1(config)# line vty 0 5

R1(config-line)# login

R1(config-line)# password cpl

R1(config-line)# exit

5. Verify Interface Status

R1# show ip interface brief

6. Test Telnet Connection from PC0

Open Command Prompt on PC0:

Ping Test

ping 192.168.1.1
(Output: success)

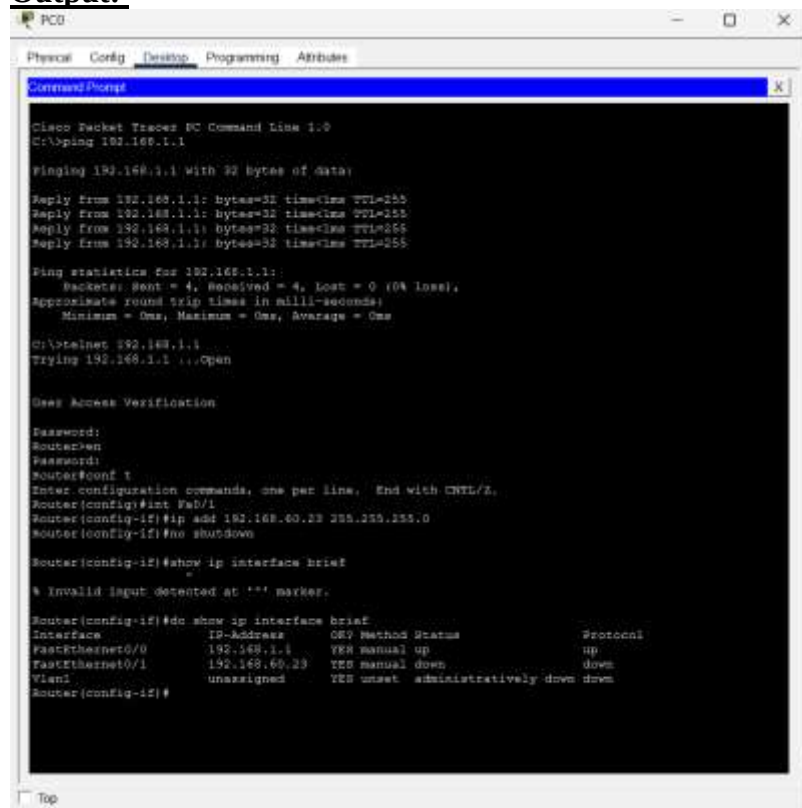
Telnet Access

telnet 192.168.1.1
password: cpl

7. Further Configuration (Second Interface)

R1> enable
Password: cpl
R1# configure terminal
R1(config)# interface Fa0/1
R1(config-if)# ip address 192.168.1.4 255.255.255.0
R1(config-if)# exit
R1# show ip interface brief

Output:



The screenshot shows a PC0 terminal window with the following content:

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:

Reply from 192.168.1.1: bytes=32 time=1ms TTL=255
Reply from 192.168.1.1: bytes=32 time=1ms TTL=255
Reply from 192.168.1.1: bytes=32 time=1ms TTL=255
Reply from 192.168.1.1: bytes=32 time=1ms TTL=255

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>telnet 192.168.1.1
Trying 192.168.1.1 ...Open

User Access Verification

Password:
Router>en
Router>conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int Fa0/1
Router(config-if)#ip add 192.168.0.23 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#show ip interface brief
% Invalid input detected at '' marker.

Router(config-if)#do show ip interface brief
Interface      IP-Address      OK? Method Status  Protocol
FastEthernet0/0 192.168.1.1     YES manual up      up
FastEthernet0/1 192.168.0.23    YES manual down  down
Vlan1          unassigned      YES unset  administratively down down
Router(config-if)#
```

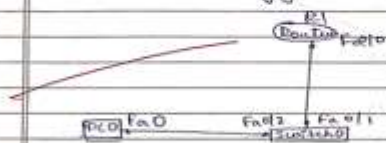
Fig 6.1 – Remote access from PC0 to router

Observation:

1. Configure TELNET to access server/router securely

- * TELNET is used to access remote server / router and it's a simple command line tool that runs on your computer and it allows you to send commands remotely to a server and administrator.
- * TELNET is also used to manage other devices like router, switch and also to check if the ports are open or close.

- Create Topology

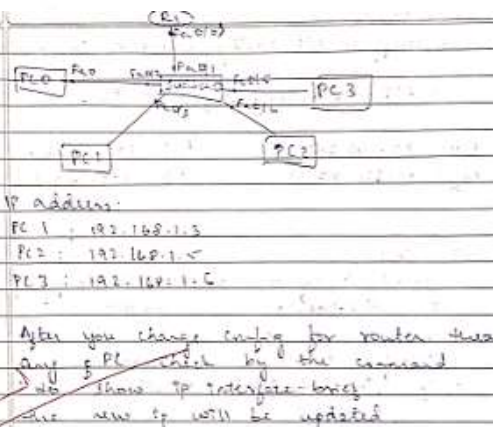


Preferences → Enable
 Configure PC0
 IP: 192.168.1.2
 Default gateway: 192.168.1.1
 Router → R1
 no (press ENTER 2 times)
 conf t
 hostname R1

enable secret 3p variable name
 int fa0/20
 ip address 192.168.1.255 255.255.255.0
 no shutdown
 (ENTER) 0 5 → 0 10
 login
 password tp
 exit
 exit
 wr (building config ok)
 show ip interface brief
 go to PC0 command prompt
 ping 192.168.1.1
 PC> telnet 192.168.1.1
 password tp → login pwd
 R1> enable
 Password sp → if configure pwd.
 R1# conf t
 # int fa0/1
 # ip address 192.168.1.4 255.255.255.0

I now connect 3 more PCs to the switch & try accessing the router through each PC

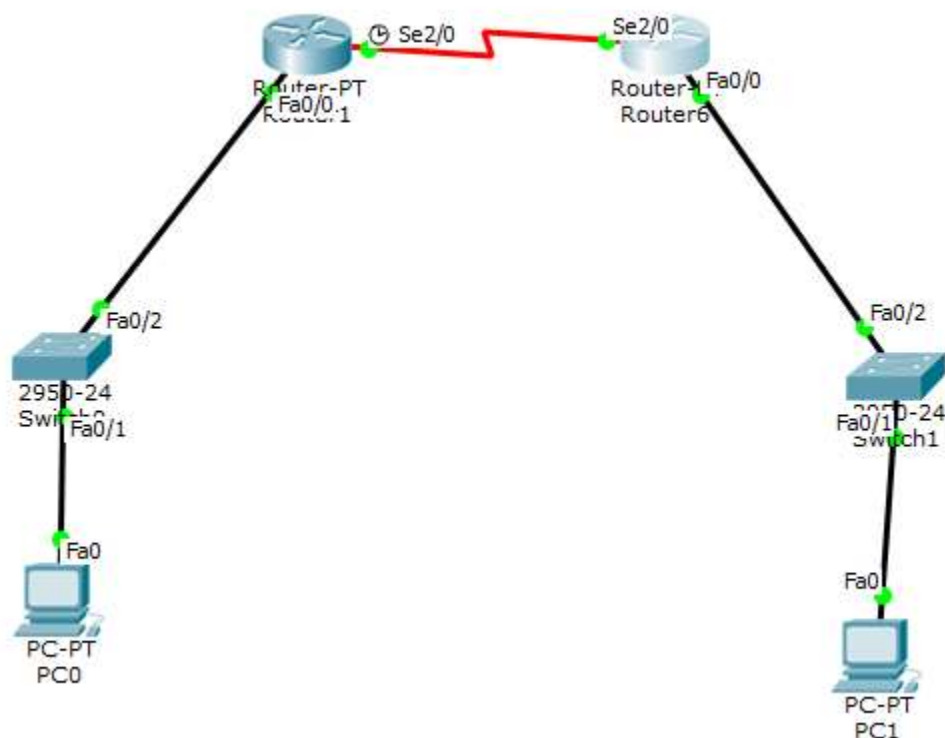
Do the same thing for different PCs. You can change the IP etc. the output on terminal will be same like fastEthernet0/0 overlaps with



Program – 7:

Aim: Configure RIP routing Protocol in Routers.

Topology:



Procedure:

1. Topology

- Two routers connected using Serial DCE.
- Router0 ↔ Router1.
- Router0 connected to Switch0 → PC0.
- Router1 connected to Switch1 → PC1.

2. PC Configuration

- **PC0**
 - IP Address: 192.168.1.2
 - Default Gateway: 192.168.1.1
- **PC1**
 - IP Address: 192.168.2.2
 - Default Gateway: 192.168.2.1

3. Router0 Configuration

- Go to CLI:
 - enable
 - configure terminal
- Configure FastEthernet0/0:
 - interface Fa0/0
 - ip address 192.168.1.1 255.255.255.0
 - no shutdown
 - exit
- Configure Serial Interface:
 - interface Se0/2
 - clock rate 64000
 - ip address 10.10.0.2 255.255.255.252

- no shutdown
 - exit
- Configure RIP:
 - router rip
 - network 192.168.1.0
 - network 10.0.0.0
 - exit
- Save:
 - write memory

4. Router1 Configuration

- Go to CLI:
 - enable
 - configure terminal
- Configure FastEthernet0/0:
 - interface Fa0/0
 - ip address 192.168.2.1 255.255.255.0
 - no shutdown
 - exit
- Configure Serial Interface:
 - interface Se0/2
 - clock rate 64000
 - ip address 10.0.0.3 255.255.255.252
 - no shutdown
 - exit
- Configure RIP:
 - router rip
 - network 192.168.2.0
 - network 10.0.0.0
 - exit
- Save:
 - write memory

5. RIP Networks Used

- Router0: 192.168.1.0 and 10.0.0.0
- Router1: 192.168.2.0 and 10.0.0.0

Output:

```

PC0
-----
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Reply from 192.168.2.2: bytes=32 time=4ms TTL=128
Reply from 192.168.2.2: bytes=32 time=4ms TTL=128
Reply from 192.168.2.2: bytes=32 time=3ms TTL=128
Reply from 192.168.2.2: bytes=32 time=4ms TTL=128

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 3ms, Maximum = 4ms, Average = 3ms

C:\>

```

Fig 7.1 Ping PC0 to PC1

Observation:

1. Configure RIP routing protocol in router

RIP → Routing Information Protocol

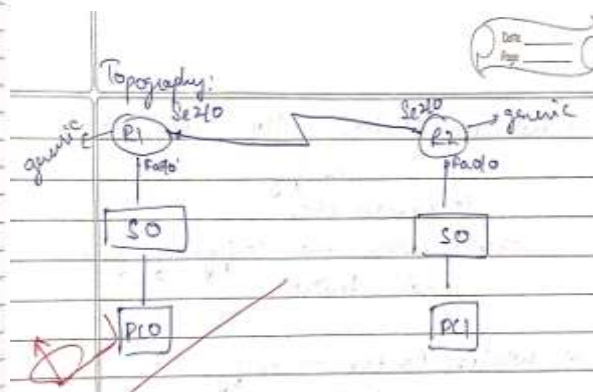
R1: 192.168.2.1 PC1: 192.168.2.2
 R2: 192.168.2.1 P2: 192.168.2.1

→ R1
 select interface Fa0/0 (in config)
 ip 192.168.2.1 → on
 R2
 interface Fa0/0 → on
 ip 192.168.2.2 192.168.2.1

→ R1 → config interface (S0/0)
 turn s/c clock to 64000
 ip 10.10.0.2
 turn on
 same for R2 ip 10.10.0.3

→ R1
 R1 192.168.2.0 add
 10.0.0.0 add
 settings → same

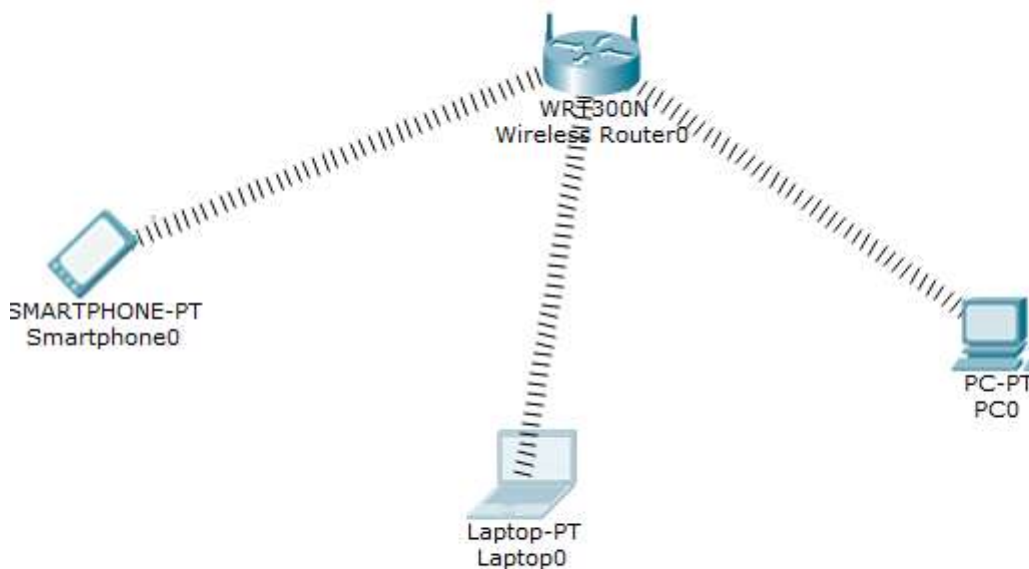
R2: R2 192.168.2.0 add
 10.0.0.0 add
 settings → same



Program – 8:

Aim: To construct a WLAN and make the nodes communicate wirelessly.

Topology:



Procedure:

1. Topology

- Wireless Router0.
- Smartphone0, Laptop0, and PC0 connected wirelessly.

2. Laptop Configuration

- Go to **Physical Tab**.
- Turn off the device.
- Remove Ethernet module.
- Add WiFi module.
- Turn device ON.

3. PC Configuration

- Go to **Physical Tab**.
- Turn off device.
- Remove Ethernet.
- Add WiFi module.
- Turn ON.

4. Wireless Router Configuration

- Go to **Config → Wireless**.
- Set **SSID: BMSCE**.
- Set **Authentication: WPA2-PSK**.
- Set **Password: BMSCE12345**.

5. Smartphone Configuration

- Enable WiFi.
- Connect to SSID **BMSCE**.
- Enter password **BMSCE12345**.

6. Connect Laptop and PC

- Open WiFi settings → select SSID **BMSCE**.
- Enter password **BMSCE12345**.

Output:

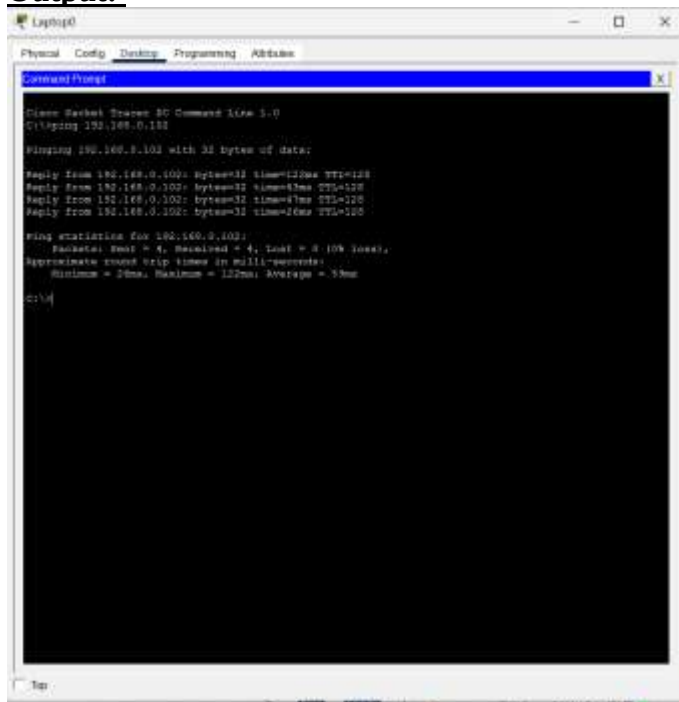
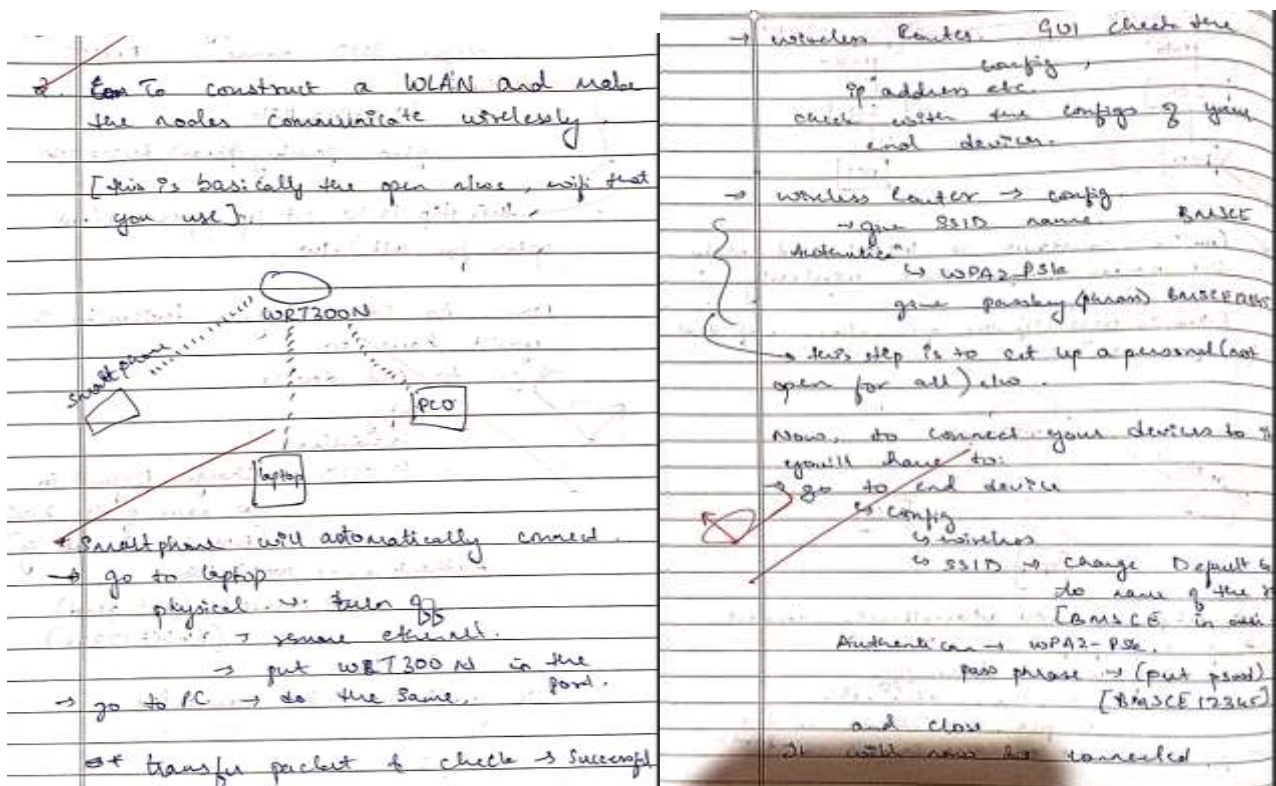


Fig 8.1 data transfer from laptop to mobile

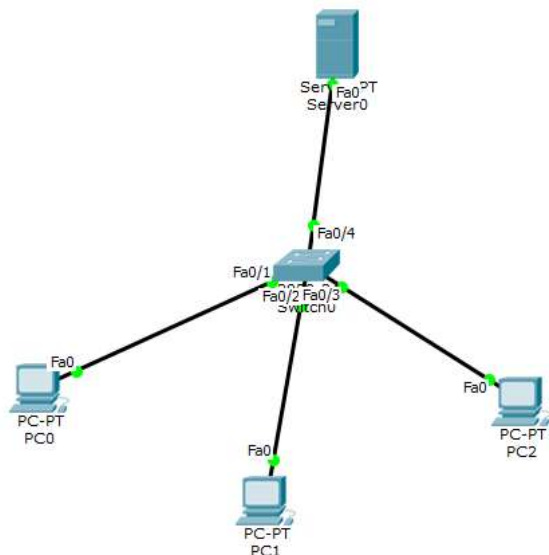
Observation:



Program – 9:

Aim: To construct simple LAN and understand the concept and operation of Address Resolution Protocol (ARP).

Topology:



Procedure:

1. ARP Concept

- ARP is used to map an IP address to a MAC address.
- ARP works at the data-link layer to resolve MAC address of a destination IP.
- It updates ARP tables with IP–MAC mappings.

2. Topology

- Server0 connected to Switch.
- Switch connected to PC0, PC1, PC2.

3. IP Configuration

- **PC0** → IP: 192.168.1.1
- **PC1** → IP: 192.168.1.2
- **PC2** → IP: 192.168.1.3
- **Server0** → IP: 192.168.1.4

4. Steps

- Open **Simulation mode** from right side.
- Click on **PC0** or **Server0** → select **ARP Table**.
- Send a packet (simulation) from PC0.
- Click **Outbound PDU Details** → view MAC address.

Output:

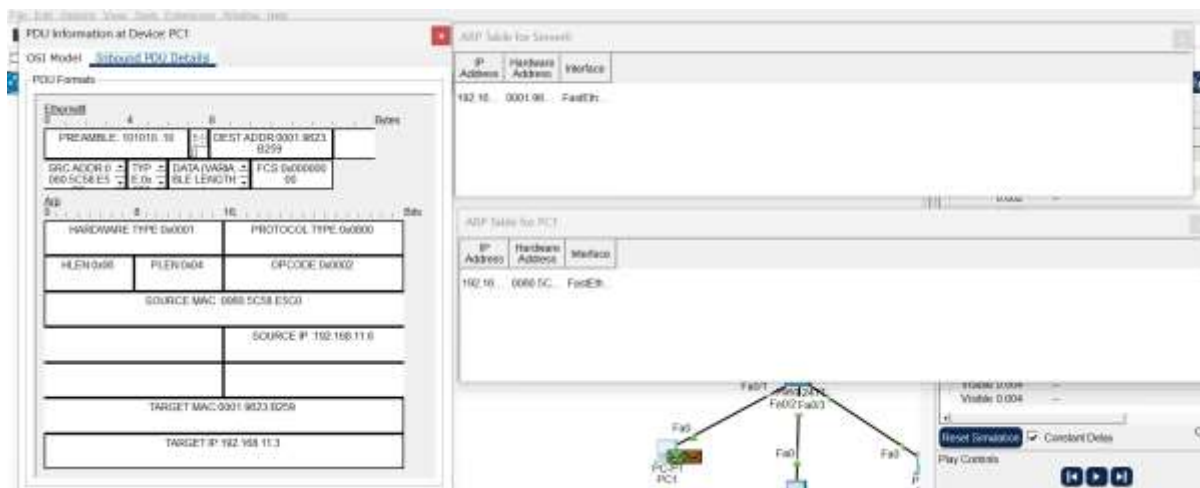


Fig 9.1 PDU Information and ARP Table of server and PC0

Observation:

3. To construct simple LAN & understand the concept of operation of ARP

ARP → Address Resolution Protocol

- ARP is used to map an IP address to a mac address
- ARP is used to get data link layer address with the help of IP address

PC0 : 192.168.11.1
PC1 : 192.168.11.2
Server 0 : 192.168.11.4

Click Sniff
→ click PC0 → ARP table
click Sniff
→ click Server → ARP table
[right now, both will be empty]

→ PC0
Command prompt
C>ARP -A
[No ARP Entries found]
C>Ping 192.168.11.4

pass packet from PC0 to Server while it is still in progress. Click on the packet & check the source & destination IP.

see the outbound etc (Auto Capture / Play)
→ Packet transfer successful

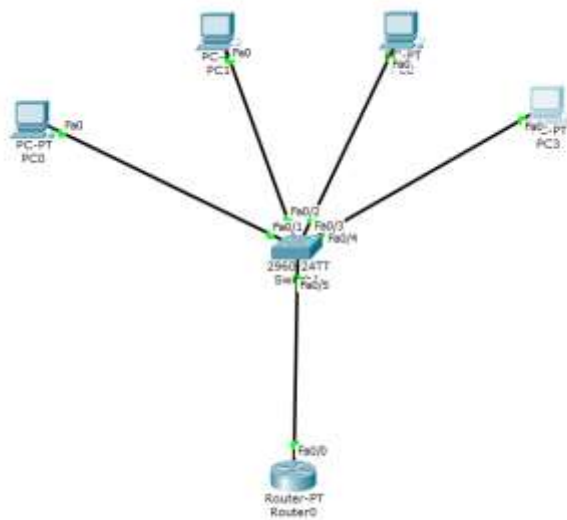
```

graph TD
    PC0[PC0 192.168.11.1] --- S0[Switch]
    PC1[PC1 192.168.11.2] --- S0
    S0 --- S1[Server0 192.168.11.4]
    S0 --- S2[PC2 192.168.11.3]
  
```


Program – 10:

Aim: To construct a VLAN and make the PC's communicate among a VLAN.

Topology:



Procedure:

1. Topology

- One Router
- One 2960-24TT Switch
- Four PCs (PC0, PC1, PC2, PC3)
- Connections using Copper Straight-Through cables

2. Router Configuration

Open CLI

- enable
- configure terminal

Configure Interfaces

Fa0/0

- interface Fa0/0
- ip address 10.0.0.1 255.0.0.0
- no shutdown
- exit

Fa0/0.1 (Sub-interface for VLAN 20)

- interface Fa0/0.1
- encapsulation dot1Q 2
- ip address 20.0.0.1 255.0.0.0
- no shutdown
- exit

3. PC Configurations

PC0

- Go to **Config** → **FastEthernet**
- IP Address: **10.0.0.2**
- Default Gateway: **10.0.0.1**

PC1

- Go to **Config** → **FastEthernet**
- IP Address: **10.0.0.3**
- Default Gateway: **10.0.0.1**

PC2

- Go to **Config** → **FastEthernet**
- IP Address: **20.0.0.3**
- Default Gateway: **20.0.0.1**

PC3

- Go to **Config** → **FastEthernet**
- IP Address: **20.0.0.2**
- Default Gateway: **20.0.0.1**

4. Switch Configuration

Create VLANs

- Go to **Config** → **VLAN Database**
- VLAN Number: **2**
- VLAN Name: **vlan**

Assign Ports to VLANs

Configure Trunk Port

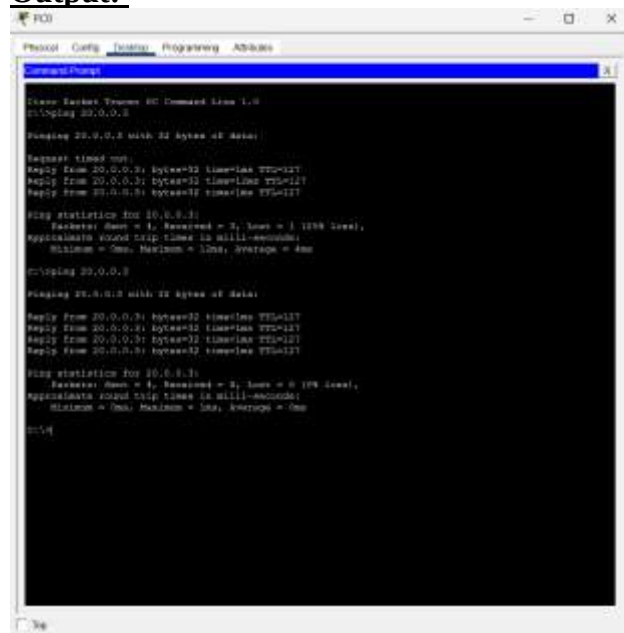
- FastEthernet0/5 (connection to router):
 - Change **Access** → **Trunk**

Assign VLAN 2 Ports

- Fa0/3 and Fa0/4 (connections to PC2 and PC3):
 - Change **VLAN** → **2**

(PC0 and PC1 remain in default VLAN 1)

Output:



```
PC0
-----
Cisco Packet Tracer PC Command Line 1.0
c:\>ping 20.0.0.2

Pinging 20.0.0.2 with 32 bytes of data:

Request timed out.
Reply from 20.0.0.2: bytes=32 time=1ms TTL=127
Reply from 20.0.0.2: bytes=32 time=1ms TTL=127
Reply from 20.0.0.2: bytes=32 time=1ms TTL=127

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

c:\>ping 20.0.0.3

Pinging 20.0.0.3 with 32 bytes of data:

Reply from 20.0.0.3: bytes=32 time=1ms TTL=127
Reply from 20.0.0.3: bytes=32 time=1ms TTL=127
Reply from 20.0.0.3: bytes=32 time=1ms TTL=127
Reply from 20.0.0.3: bytes=32 time=1ms TTL=127

Ping statistics for 20.0.0.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

c:\>
```

Fig 10.1 pinging to the other vLan -PC0 to PC2

Observation:

Aim: To Configure a VLANs and make PCs communicate among VLANs.

We use Ethernet for inter VLAN communication

Topology: 4 generic PC } copper straight
1 switch (2960) } wire
1 Router -> generic } copper straight wire

PO -> Config
C> FastEthernet 0/0 -> IP -> 10.0.0.1 -> S0/0

PC0 -> Config -> Settings

IP: 10.0.0.2

Gateway: 10.0.0.1

PC1 -> Config -> gateway 10.0.0.1

FastEthernet 0/1 10.0.0.3

PC2 -> ..

FE -> IP -> 10.0.0.4

4. Copy Switch packet -> PC0 -> PC1
Router -> PC0.

PC2: Gateway: 20.0.0.1

IP: 20.0.0.3

PC3: Gateway: 20.0.0.1

IP: 20.0.0.2

Switch -> Config -> VLAN Database

VLAN Number: 2

VLAN Name: vlan

-> ADD ✓

4. 2. port labels are not visible

-> Options -> preferences -> Always show port label.

After Check the port in first Switch (in our case PC0)
Access -> make trunk

S -> Fac0/0 | Fac0/4 (comes to the switch)
VLAN -> 2 ✓

Router -> C1

enable

config t

interface fa0/0

ip address 10.0.0.1 255.0.0.0

no shutdown

exit

interface fa0/0/1

encapsulation dot1q 2 (Set. formed to enable inter VLAN routing)

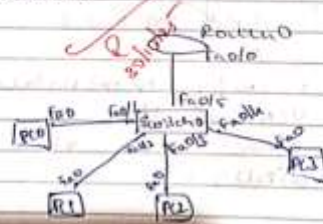
ip address 20.0.0.1 255.0.0.0

no shutdown

exit

PC0 -> Command prompt

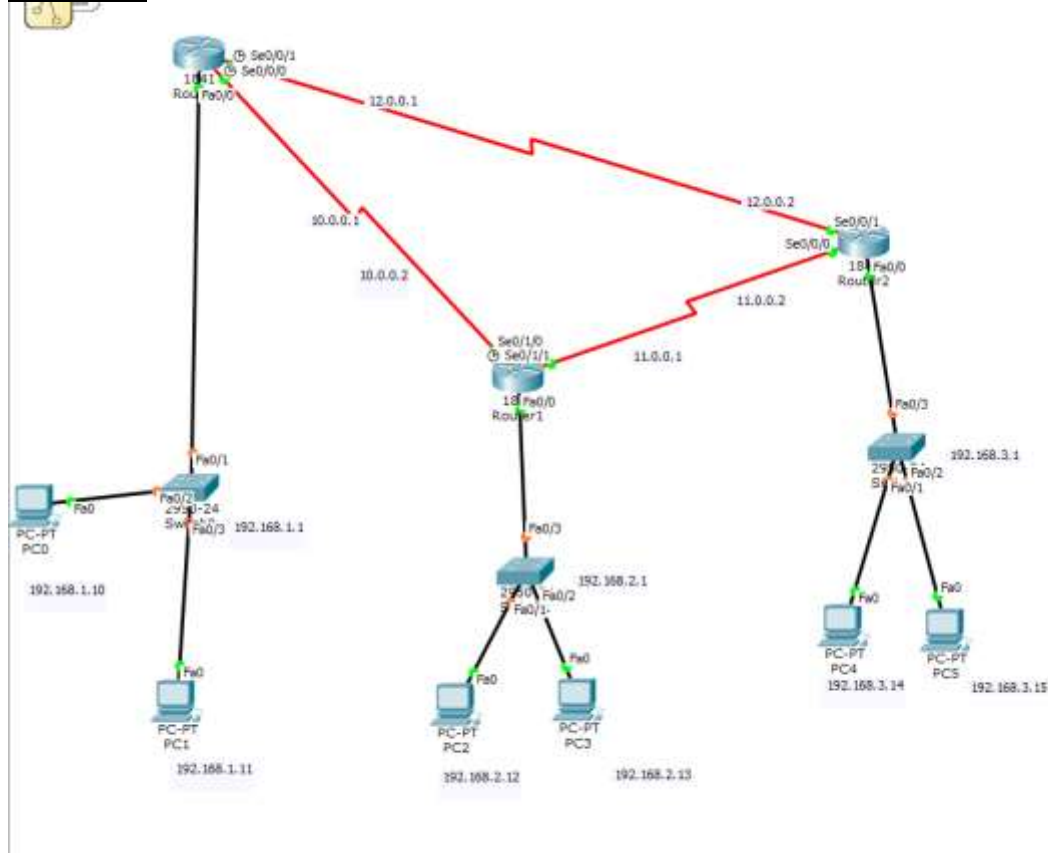
> ping 20.0.0.3



Program – 11:

Aim: Configure OSPF routing protocol.

Topology:



Procedure:

1. Topology

- Three routers R0, R1, R2 connected using Serial DCE links.
- Each router connected to its own LAN via a switch.
- PCs connected as per diagram:
 - PC0 → 192.168.1.10
 - PC1 → 192.168.1.11
 - PC2 → 192.168.2.12
 - PC3 → 192.168.2.13
 - PC4 → 192.168.3.14
 - PC5 → 192.168.3.15

2. Initial Steps

1. On each router:
 - Go to **Physical tab**
 - Turn OFF the router
 - Insert **HWIC-2T module** (2 times)
 - Turn router ON
2. Create topology using:
 - **Serial DCE** for router-to-router links
 - **Normal copper** for router-to-switch connections
3. Do IP configuration for all PCs as per diagram with switch's IP as default gateway.

3. Router Configuration

Router R0

CLI Steps

- enable
- configure terminal
- hostname R0

Fa0/0

- interface Fa0/0
- ip address 192.168.1.1 255.255.255.0
- no shutdown
- exit

Serial Se0/1/0

- interface Se0/1/0
- ip address 10.0.0.1 255.0.0.0
- clock rate 64000
- no shutdown
- exit

Serial Se0/1/1

- interface Se0/1/1
- ip address 12.0.0.1 255.0.0.0
- clock rate 64000
- no shutdown
- exit

Configure OSPF

- router ospf 1
- network 192.168.1.0 0.0.0.255 area 0
- network 10.0.0.0 0.255.255.255 area 0
- network 12.0.0.0 0.255.255.255 area 0
- exit
- wr

Router R1

CLI Steps

- enable
- configure terminal
- hostname R1

Fa0/0

- interface Fa0/0
- ip address 192.168.2.1 255.255.255.0
- no shutdown
- exit

Se0/1/0

- interface Se0/1/0
- ip address 10.0.0.2 255.0.0.0
- no shutdown
- exit

Se0/1/1

- interface Se0/1/1
- ip address 11.0.0.1 255.0.0.0
- clock rate 64000
- no shutdown
- exit

Configure OSPF

- router ospf 1
- network 192.168.2.0 0.0.0.255 area 0
- network 10.0.0.0 0.255.255.255 area 0
- network 11.0.0.0 0.255.255.255 area 0

- exit
- wr

Router R2

CLI Steps

- enable
- configure terminal
- hostname R2

Fa0/0

- interface Fa0/0
- ip address 192.168.3.1 255.255.255.0
- no shutdown
- exit

Se0/1/0

- interface Se0/1/0
- ip address 11.0.0.2 255.0.0.0
- no shutdown
- exit

Se0/1/1

- interface Se0/1/1
- ip address 12.0.0.2 255.0.0.0
- no shutdown
- exit

Configure OSPF

- router ospf 1
- network 192.168.3.0 0.0.0.255 area 0
- network 11.0.0.0 0.255.255.255 area 0
- network 12.0.0.0 0.255.255.255 area 0
- exit
- wr

Output:

```

Router#
Router>enable
Router#configure terminal
Router(config)#hostname R2
R2(config)#
R2(config)#interface Fa0/0
R2(config-if)#ip address 192.168.3.1 255.255.255.0
R2(config-if)#no shutdown
R2(config-if)#exit
R2(config)#interface Se0/1/0
R2(config-if)#ip address 11.0.0.2 255.0.0.0
R2(config-if)#no shutdown
R2(config-if)#exit
R2(config)#interface Se0/1/1
R2(config-if)#ip address 12.0.0.2 255.0.0.0
R2(config-if)#no shutdown
R2(config-if)#exit
R2(config)#router ospf 1
R2(config-router)#network 192.168.3.0 0.0.0.255 area 0
R2(config-router)#network 11.0.0.0 0.255.255.255 area 0
R2(config-router)#network 12.0.0.0 0.255.255.255 area 0
R2(config-router)#exit
R2(config)#write
R2#

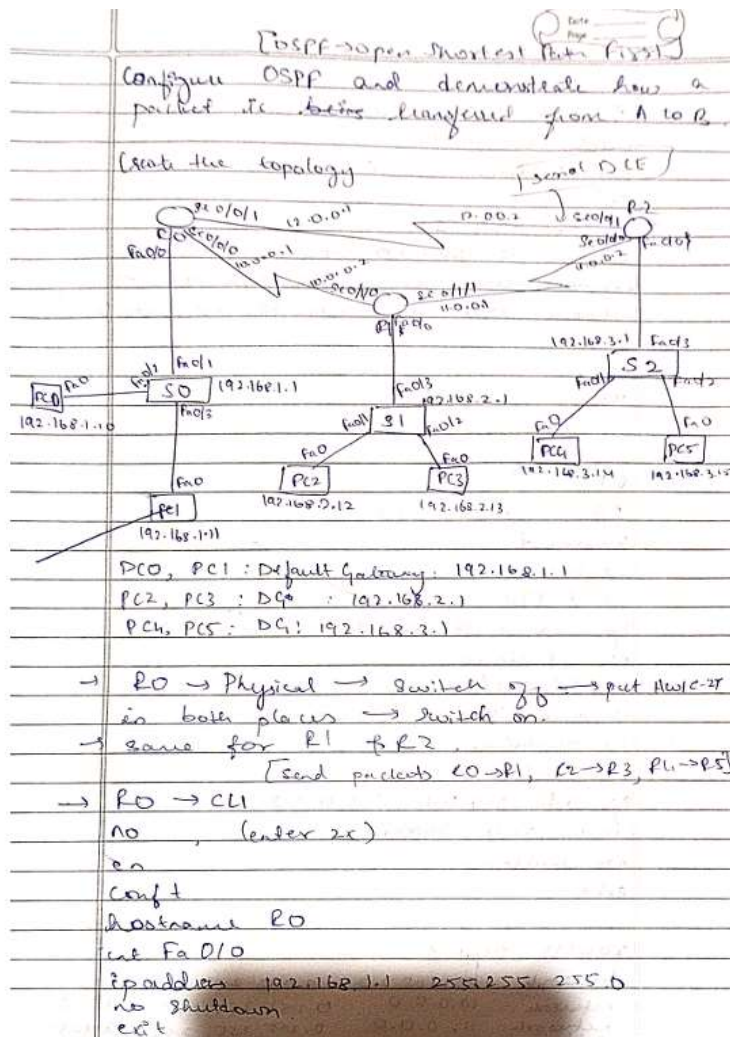
```

Fig 11.1 OSPF configuration in Router0



Fig 11.2 ping PC0 to PC3

Observation:



```

int fa0/0/1
ip add 12.0.0.1 255.0.0.0
clock rate 64000
no shutdown
exit

router ospf 1
network 192.168.1.0 0.0.0.255 area 0
network 10.0.0.0 0.255.255.255 area 0
network 12.0.0.0 0.255.255.255 area 0
exit

```

→ R1

```

no (enter 20)
conf t
hostname R1
int fa0/0
ip address 192.168.2.1 255.255.255.0
no shutdown
int se0/1/0
ip add 10.0.0.2 255.0.0.0
no shutdown
int se0/1/1
ip add 11.0.0.1 255.0.0.0
clock rate 64000
no shutdown
exit

router ospf 1
network 192.168.2.0 0.0.0.255 area 0
network 10.0.0.0 0.255.255.255 area 0
network 11.0.0.0 0.255.255.255 area 0
exit

```

→ R2

```

no (enter 20)
conf t
hostname R2
int fa0/0
ip add 192.168.3.1 255.255.255.0
no shutdown
int se0/0/0
ip add 11.0.0.2 255.0.0.0
no shutdown
int se0/0/1
ip add 12.0.0.1 255.0.0.0
no shutdown
router ospf 1
network 192.168.3.0 0.0.0.255 area 0
network 11.0.0.0 0.255.255.255 area 0
network 12.0.0.0 0.255.255.255 area 0
exit

```

→ R3, R0, R1, R2 → check by typing
show ip route
You will get a table showing the
pass packets R0 → R2 ✓
R0 → R1 ✓

Program – 12:

Aim: Write a program for congestion control using Leaky bucket algorithm.

Program:

```
def leaky_bucket(bucket_capacity, output_rate, incoming_packets):
    stored = 0 # current number of packets in the bucket

    # Print Table Header
    print(f"{'Time (s)':<12}{'Incoming Packets':<18}{'Bucket State (Before Leak)':<24}{'Dropped Packets':<15}{'Transmitted Packets':<20}{'Packets Left in Bucket'}")
    print("="*90)

    for time, packets in enumerate(incoming_packets, start=1):
        # Handle overflow: if incoming packets cause bucket overflow
        if packets + stored > bucket_capacity:
            dropped = (packets + stored) - bucket_capacity
            stored = bucket_capacity
        else:
            dropped = 0
            stored += packets

        # Transmit packets at output rate
        transmitted = min(stored, output_rate)
        stored -= transmitted

        # Print row for the table
        print(f"{'time':<12}{'packets':<18}{'stored + transmitted':<24}{'dropped':<15}{'transmitted':<20}{'stored'}")

    # Empty remaining packets in the bucket after incoming packets are done
    while stored > 0:
        time += 1
        transmitted = min(stored, output_rate)
        stored -= transmitted

        # Print row for remaining packets
        print(f"{'time':<12}{'--':<18}{'stored + transmitted':<24}{'--':<15}{'transmitted':<20}{'stored'}")

    print("\nAll packets transmitted successfully.")

# ---- Main Program ----
if __name__ == "__main__":
    bucket_capacity = int(input("Enter bucket capacity (packets): "))
    output_rate = int(input("Enter output rate (packets/sec): "))

    n = int(input("Enter number of incoming packet sets: "))
    incoming_packets = []

    for i in range(n):
        packets = int(input(f"Packets arriving at time {i + 1}: "))
        incoming_packets.append(packets)

    leaky_bucket(bucket_capacity, output_rate, incoming_packets)
```

Output:

Enter bucket capacity (packets): 5
Enter output rate (packets/sec): 1
Enter number of incoming packet sets: 5
Packets arriving at time 1: 6
Packets arriving at time 2: 4
Packets arriving at time 3: 8
Packets arriving at time 4: 1
Packets arriving at time 5: 0

Time (s)	Incoming Packets	Bucket State (Before Leak)	Dropped Packets	Transmitted Packets	Packets Left in Bucket
1	6	5	1	1	4
2	4	5	3	1	4
3	8	5	7	1	4
4	1	5	0	1	4
5	0	4	0	1	3
6	--	3	--	1	2
7	--	2	--	1	1
8	--	1	--	1	0

All packets transmitted successfully.

Observation:

Date _____
Page _____

Leaky Bucket Algorithm

```

for t=0 to T-1 do
    arrivals ← arrival[t]
    beforeQueue ← bucket

    if (bucket + arrivals ≤ bucket-size) then
        bucket ← bucket + arrivals
    else
        canAccept ← bucket-size - bucket
        if (canAccept < 0) then
            canAccept ← 0
        end if
        dropped ← dropped + (arrivals - canAccept)
        bucket ← bucket-size
    end if

    afterArrival ← bucket

    if (bucket ≥ output-rate) then
        bucket ← bucket - output-rate
        actualSent ← output-rate
    else
        actualSent ← bucket
        bucket ← 0
    end if

    sent ← sent + actualSent
end for

Print "Total packets sent = " + sent + " (at " + output-rate
"/sec)"
Print "Total packets dropped = " + dropped

```

Program – 13:

Aim: Write a program for error detecting code using CRC-CCITT (16-bits).

Program:

```
#!/usr/bin/env python3
```

```
"""
```

CRC-CCITT (16-bit) interactive demonstration.

Shows step-by-step binary long division at sender and receiver.

```
"""
```

```
def validate_binary_str(s, name="input"):
    if not s:
        raise ValueError(f"{name} cannot be empty.")
    if any(c not in '01' for c in s):
        raise ValueError(f"{name} must contain only '0' and '1' characters.")
    return s

def xor(a, b):
    """XOR two binary strings of same length, return result string"""
    return ''.join('0' if x == y else '1' for x, y in zip(a, b))

def long_division_show(dividend, divisor, show_steps=True):
    """
    Perform binary long division (modulo-2) and return remainder.
    If show_steps True, prints the division steps.
    dividend/divisor are strings of '0'/'1'. divisor's leading bit must be '1'.
    """
    n = len(divisor)
    # Work on a mutable list for the dividend bits
    work = list(dividend)
    if show_steps:
        print("\nLong division steps (divisor length = {}):".format(n))
        print("Divisor: {}".format(divisor))
        print("-" * 60)

    for i in range(len(dividend) - n + 1):
        # show current window
        window = ''.join(work[i:i+n])
        if show_steps:
            print(f"Step {i+1:02d}: position {i:02d}, window = {window}", end="")
        if window[0] == '1':
            # XOR with divisor
            new_bits = xor(window, divisor)
            if show_steps:
                print(f" -> XOR with divisor -> {new_bits}")
            # write back
            work[i:i+n] = list(new_bits)
        else:
            if show_steps:
                print(f" -> leading bit 0 -> skip (would XOR with zeros)")
            # effectively XOR with zeros; we can skip
            # (no change to work)
    remainder = ''.join(work[-(n-1):]) if n > 1 else '0'
    if show_steps:
        print("-" * 60)
```

```

    print(f"Remainder (last {n-1} bits): {remainder}\n")
    return remainder

def compute_crc_sender(data_bits, generator_bits, show_steps=True):
    """
    Compute CRC for data_bits using generator_bits.
    Returns the CRC (remainder) and the transmitted bits (data + crc).
    """
    validate_binary_str(data_bits, "Data bits")
    validate_binary_str(generator_bits, "Generator bits")
    if generator_bits[0] != '1':
        raise ValueError("Generator polynomial must start with '1' (highest-order bit).")

    k = len(generator_bits) - 1 # CRC length in bits
    # Append k zeros to data
    augmented = data_bits + '0' * k
    if show_steps:
        print("SENDER SIDE")
        print("Original data bits : {}".format(data_bits))
        print("Generator polynomial : {}".format(generator_bits))
        print("Augmented (data+0s) : {}".format(augmented))
    remainder = long_division_show(augmented, generator_bits, show_steps=show_steps)
    # CRC is remainder; transmitted frame is original data + remainder
    transmitted = data_bits + remainder
    if show_steps:
        print(f"Computed CRC (remainder) : {remainder}")
        print(f"Transmitted bits : {transmitted}\n")
    return remainder, transmitted

def check_crc_receiver(received_bits, generator_bits, show_steps=True):
    """
    Receiver side: check received_bits using generator_bits.
    Returns True if no error detected (remainder all zeros), False otherwise.
    """
    validate_binary_str(received_bits, "Received bits")
    validate_binary_str(generator_bits, "Generator bits")
    if generator_bits[0] != '1':
        raise ValueError("Generator polynomial must start with '1' (highest-order bit).")
    if show_steps:
        print("RECEIVER SIDE")
        print("Received bits : {}".format(received_bits))
        print("Generator polynomial : {}".format(generator_bits))
    remainder = long_division_show(received_bits, generator_bits, show_steps=show_steps)
    ok = all(ch == '0' for ch in remainder)
    if show_steps:
        if ok:
            print("Receiver remainder is all zeros -> No error detected.")
        else:
            print("Receiver remainder is NOT all zeros -> Error detected!")
    return ok, remainder

def demo_interactive():
    print("=== CRC-CCITT (16-bit) Demo ===")
    print("Enter data bits (binary), e.g. 11010011101100")
    data_bits = input("Data bits: ").strip()

```

```

# Default CRC-CCITT generator:  $x^{16} + x^{12} + x^5 + 1$ 
default_generator = "10001000000100001" # 17 bits (degree 16 down to 0)
print("\nPress Enter to use default CRC-CCITT generator:")
print(f" default (CRC-CCITT) = {default_generator}")
gen_in = input("Generator bits (binary) [default: ").strip()
generator_bits = gen_in if gen_in else default_generator

# Sender computes CRC and shows steps
try:
    crc, transmitted = compute_crc_sender(data_bits, generator_bits, show_steps=True)
except ValueError as e:
    print("Input error:", e)
    return

# Ask if we should flip bits (simulate errors)
print("\nDo you want to simulate transmission errors?")
print(" 1) No (send as-is)")
print(" 2) Flip one bit")
print(" 3) Flip multiple bits (specify positions)")
choice = input("Choice [1/2/3]: ").strip() or '1'

received = transmitted
if choice == '2':
    pos = input(f"Enter bit position to flip (0..{len(transmitted)-1}), 0 is leftmost: ").strip()
    try:
        p = int(pos)
        if not (0 <= p < len(transmitted)):
            raise ValueError
        b = '1' if transmitted[p] == '0' else '0'
        received = transmitted[:p] + b + transmitted[p+1:]
        print(f"Flipped bit at position {p}. Received bits: {received}")
    except:
        print("Invalid position; no flip performed.")
elif choice == '3':
    pos_list = input(f"Enter space-separated positions to flip (0..{len(transmitted)-1}): ").strip().split()
    changed = list(transmitted)
    for pos in pos_list:
        try:
            p = int(pos)
            if 0 <= p < len(transmitted):
                changed[p] = '1' if changed[p] == '0' else '0'
        except:
            pass
    received = "".join(changed)
    print(f"Received bits after flips: {received}")
else:
    print("Sending without errors.")

# Receiver checks CRC
ok, rem = check_crc_receiver(received, generator_bits, show_steps=True)
if ok:
    print("\nFINAL RESULT: No error detected by CRC.")
else:
    print("\nFINAL RESULT: Error detected by CRC.")

```



```
if __name__ == "__main__":
    demo_interactive()
```

Output:

```
... default (CRC-CCITT) = 10001000000100001
Generator bits (binary) [default]:
SENDER SIDE
Original data bits : 10110101011
Generator polynomial : 10001000000100001
Augmented (data+0s) : 10110101100000000000000000

Long division steps (divisor length = 17):
Divisor: 10001000000100001
-----
Step 01: position 00, window = 101101011000000 -> XOR with divisor -> 001111011100001
Step 02: position 01, window = 0111101011000010 -> leading bit 0 -> skip (would XOR with zeros)
Step 03: position 02, window = 11110101110000100 -> XOR with divisor -> 0111101110100101
Step 04: position 03, window = 1111011101001010 -> XOR with divisor -> 0111001110110101
Step 05: position 04, window = 11100111011010110 -> XOR with divisor -> 0110111101110111
Step 06: position 05, window = 11011110111101110 -> XOR with divisor -> 01010110111001111
Step 07: position 06, window = 10101101110011110 -> XOR with divisor -> 00100101110111111
Step 08: position 07, window = 0100101110111110 -> leading bit 0 -> skip (would XOR with zeros)
Step 09: position 08, window = 10010111011111100 -> XOR with divisor -> 00011111011011101
Step 10: position 09, window = 0011110110111010 -> leading bit 0 -> skip (would XOR with zeros)
Step 11: position 10, window = 0111101101110100 -> leading bit 0 -> skip (would XOR with zeros)
-----
Remainder (last 16 bits): 111110110110100

Computed CRC (remainder) : 111110110110100
Transmitted bits : 1011010101111110110110100

Do you want to simulate transmission errors?
1) No (send as-is)
2) Flip one bit
3) Flip multiple bits (specify positions)
Choice [1/2/3]: 1
Sending without errors.
RECEIVER SIDE
Received bits : 1011010101111110110110100
Generator polynomial : 10001000000100001

Long division steps (divisor length = 17):
Divisor: 10001000000100001
-----
Step 01: position 00, window = 1011010101111110 -> XOR with divisor -> 00111101101101111
Step 02: position 01, window = 0111101011011111 -> leading bit 0 -> skip (would XOR with zeros)
Step 03: position 02, window = 1111010110111111 -> XOR with divisor -> 0111101101011110
Step 04: position 03, window = 1111011010111100 -> XOR with divisor -> 01110011010011101
Step 05: position 04, window = 11100110100111011 -> XOR with divisor -> 01101110100011010
Step 06: position 05, window = 11011101000110101 -> XOR with divisor -> 01010101000010100
Step 07: position 06, window = 10101010000101001 -> XOR with divisor -> 00100010000001000
Step 08: position 07, window = 01000100000010000 -> leading bit 0 -> skip (would XOR with zeros)
Step 09: position 08, window = 10001000000100001 -> XOR with divisor -> 00000000000000000
Step 10: position 09, window = 0000000000000000 -> leading bit 0 -> skip (would XOR with zeros)
Step 11: position 10, window = 0000000000000000 -> leading bit 0 -> skip (would XOR with zeros)
-----
Remainder (last 16 bits): 0000000000000000

Receiver remainder is all zeros -> No error detected.

FINAL RESULT: No error detected by CRC
```

Observation:

Algorithm for Cyclic Redundancy Check

1. Input data bits [], generator bits []
2. Append zeros = for $i = 1$ to $\text{len}(\text{generator bits})$
data bits + 0
3. temp = data bits
4. for $i = 0$ to $\text{len}(\text{temp}) - \text{len}(\text{generator bits})$
if temp[i] == '1':
for $j = 0$ to $\text{len}(\text{generator bits}) - 1$
temp[i+j] = XOR(temp[i+j], generator bits[j])
5. ~~remainder~~
5. remainder = last $(\text{len}(\text{generator bits}) - 1)$ bits of temp
6. Transmitted = data bits + remainder
7. at receiver
repeat step 4 on transmitted
if remainder == 0 → OK
else → error

Output

Enter data bits : 101101

Enter generator bits : 1101

CRC bits : 010

Transmitted data (data + CRC) = 101101010

No error detected : Data received correct

Program – 14:

Aim: Using TCP/IP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.

Program:

CLIENTSIDE:

```
package pgm7;
import java.net.*;
import java.io.*;

public class TCPC {
    public static void main(String[] args) throws Exception {
        Socket sock = new Socket("127.0.0.1", 4000);

        System.out.println("Enter the filename:");
        BufferedReader keyRead = new BufferedReader(new InputStreamReader(System.in));

        String fname = keyRead.readLine();

        OutputStream ostream = sock.getOutputStream();
        PrintWriter pwrite = new PrintWriter(ostream, true);

        // Send file name to server
        pwrite.println(fname);

        InputStream istream = sock.getInputStream();
        BufferedReader socketRead = new BufferedReader(new InputStreamReader(istream));

        String str;
        while ((str = socketRead.readLine()) != null) {
            System.out.println(str);
        }

        pwrite.close();
        socketRead.close();
        keyRead.close();
        sock.close();
    }
}
```

```
import java.net.*;
import java.io.*;
```

```
public class TCPS {
    public static void main(String[] args) throws Exception {

        ServerSocket sersock = new ServerSocket(4000);
        System.out.println("Server ready for connection...");

        Socket sock = sersock.accept();
        System.out.println("Connection successful. Waiting for filename...");

        InputStream istream = sock.getInputStream();
        BufferedReader fileRead = new BufferedReader(new InputStreamReader(istream));
    }
}
```

```

String fname = fileRead.readLine(); // Read filename from client
BufferedReader contentRead = new BufferedReader(new FileReader(fname));

OutputStream ostream = sock.getOutputStream();
PrintWriter pwrite = new PrintWriter(ostream, true);

String str;
while ((str = contentRead.readLine()) != null) {
    pwrite.println(str);
}

sock.close();
sersock.close();
pwrite.close();
fileRead.close();
contentRead.close();
}
}

```

Output:

SERVER OUTPUT:

```

Server ready for connection...
Connection successful. Waiting for filename...

```

CLIENT OUTPUT:

```

----- CLIENT SIDE -----
Enter the filename
test.txt

Hello World
This is a TCP Socket File Transfer Program
End of File
~

```

Observation:

Using TCP/IP sockets, write a client server prog. to make client sending the file name and the server to send back the contents of the requested file if present.

```
import java.io.*;
import java.net.*;

public class client {
    public static void main (String args[]) throws
        Exception {
        Socket sock = new Socket ("127.0.0.1", 6005);
        System.out.println ("Enter the filename");
        BufferedReader keyboard = new BufferedReader
            (new InputStreamReader(System.in));
        String fname = keyboard.readLine();
        OutputStream ostream = sock.getOutputStream();
        PrintWriter pwrite = new PrintWriter (ostream, true);
        pwrite.println(fname);
        InputStream istream = sock.getInputStream();
        String str;
        while ((str = istream.readLine()) != null) {
            System.out.println (str);
        }
        pwrite.close();
        sock.close();
        keyboard.close();
    }
}
```

server code -

```
import java.io.*;
import java.net.*;

public class server {
    public static void main (String args[]) throws
        Exception {
        ServerSocket ssock = new ServerSocket(6005);
        System.out.println ("Server ready for connection");
        Socket sock = ssock.accept();
        System.out.println ("Connection successful,
            waiting for chatting");
        InputStream istream = sock.getInputStream();
        String fname = keyboard.readLine();
        BufferedReader contentRead = new BufferedReader
            (new FileReader(fname));
        OutputStream ostream = sock.getOutputStream();
        PrintWriter pwrite = new PrintWriter (ostream, true);
        String str;
        while ((str = contentRead.readLine()) != null) {
            pwrite.println (str);
        }
    }
}
```

Output:
Server ready for connection
connection successful, waiting for chat
Enter the filename
delta.txt
hi

Program – 15:

Aim: Using UDP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.

Program:

```
package pgm8;

import java.io.*;
import java.net.*;

class UDPClient {
    public static void main(String[] args) throws Exception {

        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));

        DatagramSocket clientSocket = new DatagramSocket();
        InetAddress IPAddress = InetAddress.getByName("localhost");

        byte[] sendData = new byte[1024];
        byte[] receiveData = new byte[1024];

        System.out.println("Enter the string to be converted to Uppercase:");
        String sentence = inFromUser.readLine();

        sendData = sentence.getBytes();

        DatagramPacket sendPacket =
            new DatagramPacket(sendData, sendData.length, IPAddress, 9876);

        clientSocket.send(sendPacket);

        DatagramPacket receivePacket =
            new DatagramPacket(receiveData, receiveData.length);

        clientSocket.receive(receivePacket);

        String modifiedSentence = new String(receivePacket.getData());

        System.out.println("FROM SERVER: " + modifiedSentence);

        clientSocket.close();
    }
}

package pgm8;

import java.net.*;

class UDPServer {
    public static void main(String[] args) throws Exception {

        DatagramSocket serverSocket = new DatagramSocket(9876);

        byte[] receiveData = new byte[1024];
```

```

byte[] sendData = new byte[1024];

while (true) {
    System.out.println("Server is Up");

    DatagramPacket receivePacket =
        new DatagramPacket(receiveData, receiveData.length);

    serverSocket.receive(receivePacket);

    String sentence = new String(receivePacket.getData());
    System.out.println("RECEIVED: " + sentence.trim());

    InetAddress IPAddress = receivePacket.getAddress();
    int port = receivePacket.getPort();

    String capitalizedSentence = sentence.toUpperCase();

    sendData = capitalizedSentence.getBytes();

    DatagramPacket sendPacket =
        new DatagramPacket(sendData, sendData.length, IPAddress, port);

    serverSocket.send(sendPacket);
}
}
}

```

Output:

```

----- SERVER SIDE -----
Server is Up
RECEIVED: hello world
Server is Up

----- CLIENT SIDE -----
Enter the string to be converted to Uppercase:
hello world
FROM SERVER: HELLO WORLD

```


Observation:

Using UDP sockets write a client-server program to make client sending the file name and the server to send back the contents of file if present.

```
4 client.java
import java.io.*;
import java.net.*;
class Client {
    public static void main (String arg[]) throws
        Exception {
        BufferedReader inFromUser = new BufferedReader
            (new InputStreamReader(System.in));
        DatagramSocket clientSocket = new DatagramSocket();
        InetAddress IPAddress = InetAddress.getByName
            ("localhost");

        byte[] sendData = new byte[1024];
        byte[] receiveData = new byte[1024];
        System.out.println("Enter string to be converted
            to uppercase");
        String str = inFromUser.readLine();
        sendData = str.getBytes();
        DatagramPacket sendPacket = new DatagramPacket
            (sendData, sendData.length, IPAddress,
            9876);

        clientSocket.send(sendPacket);
        DatagramPacket receivePacket = new DatagramPacket
            (receiveData, receiveData.length);
        clientSocket.receive(receivePacket);
    }
}
```

```
1 server.java
import java.net.*;
class Server {
    public static void main (String arg[]) throws
        Exception {
        DatagramSocket serverSocket = new DatagramSocket
            (9876);

        byte[] receiveData = new byte[1024];
        byte[] sendData = new byte[1024];
        while(true) {
            System.out.println("Server loop");
            DatagramPacket receivePacket = new
                DatagramPacket(receiveData, receiveData.length);
            serverSocket.receive(receivePacket);
            String str = new String(receivePacket.getData());
            System.out.println("Received: " + str);
            InetAddress IPAddr = receivePacket.getAddress();
            int port = receivePacket.getPort();
            String CapitalizedSentence = str.toUpperCase();
            sendData = CapitalizedSentence.getBytes();
            DatagramPacket sendPacket = new DatagramPacket
                (sendData, sendData.length, IPAddr,
                port);
            serverSocket.send(sendPacket);
        }
    }
}
```

Output:

Server is up
Received input my luck
Server is up

Enter the string to be converted to
Upper Case
input my luck
from server: JUST MY LUCK